

Análisis de la Función Hash en Java

Universidad Distrital Francisco José de Caldas

Juan David Roa Ahumada

Juan David Sabogal Arenas

Jose Luis Leudo Mosquera

Docente:

Simar Enrique Herrera Jiménez

1. Introducción

El presente informe analiza el código fuente Hashing.java, una implementación en Java de una función hash personalizada (ad-hoc), es decir, no corresponde a un algoritmo estándar de la industria (como MD5, SHA-1 o SHA-256), sino a un diseño propio con fines educativos, que combina operaciones de conversión binaria, segmentación de bloques y desplazamientos mediante XOR (OR exclusivo).

El objetivo del programa es tomar una palabra ingresada por el usuario y transformarla en una cadena hexadecimal de longitud fija que actúa como "huella digital" (hash) del texto original.

2. Descripción General del Algoritmo

El proceso se ejecuta en 6 pasos principales, orquestados por el método Hash():

1. Conversión a binario (32 bits por carácter) — `convertir32Bytes()`
2. División en bloques de 16 bits — `dividirArreglo()`
3. Mezcla del segundo bloque — `cambioSegundoElemento()`
4. Encadenamiento de bloques (desmadre + XOR) — `encadenarBloques()`, que usa `desmadreBloque()` y `xorBloques()`
5. Segmentación del resultado final en bloques de 16 bits
6. Conversión a hexadecimal — `convertiraTexto()`

3. Análisis Detallado por Componente

3.1 `convertir32Bytes()`

Lee una palabra por teclado (Scanner) y, para cada carácter:

- Obtiene su valor ASCII `((int) letra)`.
- Lo convierte a binario con `Integer.toBinaryString()`.
- Rellena el binario hasta 32 bits usando `String.format("%32s", binario).replace(' ', '1')`.

3.2 dividirArreglo(StringBuilder sb, int tamanoBloque)

Divide la cadena de bits completa en bloques de tamaño fijo (16 en este caso), calculando la cantidad de bloques con `Math.ceil()`. Es una función reutilizada dos veces en el flujo: primero sobre la cadena de 32 bits por carácter, y después sobre el resultado final del encadenamiento.

Riesgo: si la longitud total no es múltiplo exacto de `tamanoBloque`, el último bloque queda incompleto (más corto), lo cual puede causar comportamientos inesperados en `xorBloques()` o en `convertiraTexto()` al procesar bloques de distinta longitud.

3.3 cambioSegundoElemento(String[] list) y desmadreBloque(String bloque)

Ambas funciones realizan la misma operación: aplican un XOR entre cada bit y el bit siguiente dentro del mismo bloque (`letras.get(i) ^ letras.get(i+1)`), dejando el último carácter del bloque sin modificar (el bucle llega hasta `size()-1`).

`desmadreBloque` es la versión generalizada y reutilizable de `cambioSegundoElemento`, pensada para aplicarse a cualquier bloque, no solo al segundo.

Riesgo: al tratarse de una operación XOR entre caracteres '0'/'1' convertidos con `Integer.parseInt()`, si algún bloque contuviera un carácter distinto de '0' o '1' se produciría una `NumberFormatException`.

3.4 xorBloques(String bloqueA, String bloqueB)

Realiza un XOR bit a bit entre dos bloques de cadena binaria, comparando carácter por carácter. Si los bloques tienen longitudes distintas, usa la longitud menor (`Math.min()`), truncando silenciosamente el bloque más largo sin advertencia.

3.5 encadenarBloques(String[] bloques)

Es el núcleo del "mezclado" del hash:

- Aplica `desmadreBloque` al primer bloque.
- Para cada bloque siguiente, hace XOR con el acumulado y, si quedan más bloques, vuelve a aplicar `desmadreBloque` al resultado.
- Concatena todos los resultados intermedios en una única cadena de salida.

Este método imprime en consola cada paso intermedio (útil para depuración y para fines didácticos, pero no recomendable en una versión de producción).

3.6 convertiraTexto(String[] cadena)

Convierte cada bloque binario final (de 16 bits) a un entero sin signo (`Integer.parseUnsignedInt(bloque, 2)`) y lo representa como dos dígitos

hexadecimales (%02x), concatenando el resultado. Este es el paso que produce la representación final legible del hash.

3.7 Hash() y main()

Hash() orquesta toda la secuencia de pasos descrita y devuelve el resultado hexadecimal final. main() simplemente invoca Hash() y lo imprime en consola.

4. Evaluación como Función Hash

Criterio	Cumplimiento	Observación
Longitud de salida fija	Parcial	Depende de la cantidad de bloques resultante; no está garantizada una longitud constante para entradas de distinto tamaño.
Efecto avalancha (pequeño cambio de entrada → gran cambio de salida)	Bajo	Las operaciones XOR y de mezcla son deterministas y simples, sin difusión suficiente.
Resistencia a colisiones	Muy baja	No hay fundamento criptográfico; es fácil encontrar entradas distintas con salidas similares o iguales.
Uso en seguridad informática	No recomendado	Es un ejercicio académico, no un algoritmo criptográfico validado.
Legibilidad y fines didácticos	Alto	El código está bien comentado y estructurado en pasos claros, ideal para enseñar conceptos de manipulación de bits.

5. Conclusión

Hashing.java es una implementación educativa que ilustra, de forma práctica, conceptos fundamentales del diseño de funciones hash: conversión de datos a binario, segmentación en bloques, mezcla mediante operaciones bit a bit (XOR) y codificación final en hexadecimal. Sin embargo, no está diseñada para ofrecer las garantías criptográficas (resistencia a colisiones, efecto avalancha, uniformidad de distribución) que se esperan de una función hash de uso real, por lo que su valor es principalmente pedagógico.